



**Videojuego inspirado en dwarf fortress
y clash of clans**

Fecha: 31/10/25

Documento para desarrolladores

**Elaborado por: David Esparza, Alejandro
Bugarin, Diego Enríquez**

Introducción

Propósito del sistema:

Simular una colonia de enanos que toman decisiones de manera autónoma, con un sistema basado en estructuras de datos personalizadas (colas, heaps, listas enlazadas y deque). El jugador solo da órdenes generales; los enanos las priorizan y ejecutan según su oficio, estado y recursos disponibles.

Alcance:

El sistema abarca gestión de tareas, planificación automática, recolección de recursos, construcción, defensa ante enemigos y una GUI animada que representa los eventos en tiempo real.

Audiencia prevista:

Desarrolladores y programadores interesados en entender la arquitectura interna del proyecto, su lógica de IA, y cómo extender o mantener el sistema.

Definiciones / acrónimos:

- **Heap / PriorityQueue:** estructura para ordenar tareas según prioridad.
- **Deque:** estructura para rotar enanos disponibles.
- **GUI:** interfaz visual en Pygame.
- **EDA:** estructura de datos abstracta.

Visión general / arquitectura

Módulos del sistema:

1. **personajes:** define todos los personajes, enemigos y cofres.
2. **planificador:** administra las tareas mediante Heap y Queue.
3. **gestion:** controla los enanos disponibles, ocupados y descansando.
4. **prueba_estres:** realiza simulaciones automáticas de 50+ acciones.
5. **Deque (Cola de Doble Extremo):** Manejo eficiente de enanos disponibles
6. **LinkedList (Lista Enlazada):** Organiza enanos ocupados y enemigos
7. **Queue (Cola FIFO):** Procesa acciones y recursos en orden
8. **Heap (Min-Heap):** Sistema de prioridades para asignación óptima de tareas
9. **main:** (en GUI) integra la simulación y las animaciones.

10. **Selector de personajes:** dependiendo de la vida (menos de 70) y del nivel de los personajes es el que selecciona para hacer la acción correspondida

Diagrama de componentes:

Jugador → Buffer (Queue) → Heap (Planificador) → Selector de Personajes → Gestor → Personajes → Cofres / Recursos → GUI

Flujo de datos:

- Las órdenes del jugador entran a una cola FIFO.
- El planificador calcula prioridad (Heap).
- El selector de personajes elige al mejor enano disponible según vida y nivel.
- El gestor asigna personajes según disponibilidad.
- Los personajes ejecutan la acción.
- Los cofres y el estado global se actualizan.
- La GUI refleja los cambios.

Diseño detallado de clases

Principales clases y métodos:

- **Personaje:** clase base con atributos de energía, hambre, estado y oficio.
- **Subclases:** Enano, Minero, Leñador, Granjero, Constructor.
- **Duende:** enemigo con ataque aleatorio.
- **CofrePrincipal, CofreAlimentos, Bar:** manejo de recursos y bebidas.
- **PlanificadorTareas:** usa Queue y Heap para priorizar tareas.
- **SelectorDePersonajes:** analiza a todos los personajes y elige al más apto para cada tarea según su vida (>70), nivel de experiencia y estado actual (ocupado o disponible).
- **GestorPoblacion:** usa Deque y LinkedList para manejar estados de personajes.

Herencia y polimorfismo:

Cada clase de personaje redefine métodos (minar, talar, cosechar, defender, etc.) según su oficio y compatibilidad con las acciones.

Interfaz/API expuesta:

Los módulos exponen métodos controlados (registrar_evento, procesar_buffer, obtener_siguiente_tarea) que la GUI usa para conectar la lógica con las animaciones.

Estructuras de datos y algoritmos

- **Cola (Queue):** administra órdenes de usuario.
 - Complejidad: Enqueue/Dequeue $\rightarrow O(1)$.
- **Heap (QHeap):** gestiona prioridades.
 - Complejidad: Insert/Extract $\rightarrow O(\log n)$.
- **Deque (Deque):** rota enanos disponibles.
 - Complejidad: $O(1)$ por operación.
- **Lista enlazada (LinkedList):** guarda enanos ocupados y descansando.
 - Complejidad: Búsqueda $O(n)$, inserción $O(1)$.

Algoritmo de decisión:

La prioridad se calcula con la fórmula:

$$\text{Prioridad} = (\text{UrgenciaEvento} * 10) + (\text{UrgenciaRecursos} * 5) + (\text{Disponibilidad} * 3)$$

Si hay empate, se desempata por disponibilidad de enanos.

Selección de objetivo:

Cada acción se asigna al personaje con mayor compatibilidad y energía disponible.

Integración GUI - lógica

- La GUI ejecuta tick() $\rightarrow$ llama al planificador $\rightarrow$ ejecuta tareas activas $\rightarrow$ actualiza sprites.
- Las animaciones reflejan estados (trabajando, descansando, peleando).
- El render muestra cofres, aldeanos, enemigos y estructuras.

Requisitos funcionales / no funcionales

Funcionales:

- Recolectar recursos (madera, piedra, comida, leche, huevos).

- Construir estructuras y reparar.
- Defender la aldea.
- Animaciones de mínimo 6 frames por acción.

No funcionales:

- Efectos de sonido y música ambiental adaptativa.
- Incluir menú de opciones gráficas (resolución, volumen, brillo).
- Soporte para partidas guardadas y carga de progreso.
- Animaciones adicionales y expresiones faciales.
- Nuevos tipos de estructuras y recursos.
- Expansión de IA enemiga con patrones más complejos.

Pruebas y métricas

Plan:

El módulo prueba_estres ejecuta 50+ acciones en secuencia: minar, talar, construir, cosechar y defender. También simula ataques aleatorios y mide estabilidad.

Métricas:

- Acciones completadas sin fallos ≥ 50 .
- Tiempo de ejecución promedio por ciclo.
- Recursos recolectados por personaje.

Resultados esperados:

- Ningún crash o bucle infinito.
- Priorización correcta de eventos urgentes.
- Animaciones coherentes con los estados del personaje.

Decisiones de diseño y riesgos

Justificación:

Se eligió Heap para priorizar eventos dinámicos. Deque para rotar personajes disponibles rápidamente y Lista enlazada para manejar ocupados sin reordenar estructuras.

Riesgos y mitigación:

- **Riesgo:** colisiones no detectadas entre los elementos del proyecto
Mitigación: sensores de contacto en los personajes, estructuras, animales, enemigos, etc.
- **Riesgo:** exceso de tareas pendientes → **Mitigación:** limpieza periódica del heap.
- **Riesgo:** falta de recursos → **Mitigación:** eventos automáticos de recolección.

Apéndices

Glosario:

- Tick: unidad de tiempo en la simulación.
- Cofre: contenedor de recursos globales.
- Evento: acción generada por usuario o ataque enemigo.

Evolución futura:

- Integrar sistema climático.
- Añadir animales domesticables.
- Implementar edificios funcionales (taller, mina, taberna).

Diagramas extra:

- Flujo de acciones (Buffer → Heap → Gestor → Personaje → Cofre).
- Jerarquía de clases (Personaje base → subclases especializadas).